

8 Scripting and System Administration

There are few areas of coding where AI support—used correctly!—works as well as when you use Python or Bash for scripting. In this context, the nature of scripts plays a crucial role. Often, relatively few lines of code are sufficient to complete a clearly defined task. While the code for large programming projects is spread across dozens, often hundreds of files, a script can often be formulated in a single file. Accordingly, there are hardly any dependencies or context issues that overwhelm AI language models.

We had actually expected to have similarly good experiences with scripting on Windows using PowerShell. But the result was rather disappointing. The GPT-4o model used in our tests delivered (as of mid-2024) much poorer quality code with significantly more errors. Of course, there is less training material for PowerShell available on the internet than for Python, but it was surprising that the difference would be so obvious.

Nevertheless, in this chapter, we present a whole range of examples where ChatGPT, GitHub Copilot, and other AI tools have helped us. Once again, our aim is to present you with a wide range of possible applications. Perhaps there is an idea or two that you haven't thought of yet?

In this chapter, we also go a little beyond the scripting horizon and look at secondary topics such as regular expressions and system administration.

System administration isn't really a classic coding topic and is therefore outside the focus of this book. But in real life, many developers and programmers are also responsible for setting up Docker containers, Git repositories, database systems, and servers. ChatGPT and other AI tools are perfectly capable of helping with this.

8.1 Scripting

From our point of view, the ideal working environment for writing new scripts is an editor with AI support (e.g., Visual Studio Code [VS Code] plus GitHub Copilot) and a dedicated chat window—whether as part of the editor or separate from it in a web browser. The AI chat provides help with the basic design of parts of the script and with researching the commands, cmdlets, or modules used. The editor supports code input, documentation, and debugging. The following pages present a colorful mix of examples.

8.1.1 Bash: Loop Over a Text File

You want to process a text file in a Bash script and skip the first line with the column label. What is the easiest way? You ask an AI tool!

Prompt: Bash: I want to loop over all lines of file but skip the first line.

In Bash, if you want to loop over all lines of a file but skip the first line, you can use the following script:

```
# Assuming 'file.txt' is your file
tail -n +2 file.txt | while IFS= read -r line
do
    echo "$line"
done
```

Here's a breakdown of the script [more details]

ChatGPT provides a solution that is as short as it is ideal. The rather lengthy explanations of the script are primarily of interest to Bash beginners.

8.1.2 PowerShell: Sorting Photos

You want to use a PowerShell script to analyze the exchangeable image file format (EXIF) data of an entire collection of photos. The image files should be moved accordingly to directories named yyyy-mm, such as 2024-06 for photos shot in June 2024.

Prompt: PowerShell script: I have a lot of JPEG files to organize. I want to read the EXIF date and then move the files to folders like yyyy-mm (year/month). Files without EXIF data should be moved to a special folder. Can you help me?

Sure! Here is a PowerShell script that reads the EXIF date from JPEG files and organizes them into folders based on the year and month. Files without EXIF data will be moved to a separate folder.

Script and explanation . . .

ChatGPT provides an approximately 60-line script that uses the `System.Drawing` system library to analyze the EXIF data. Although this solution is relatively cumbersome, it has one major advantage: no additional tools or libraries need to be installed.

However, an initial test run fails: the script opens each image file for EXIF analysis, but doesn't close the file again. This is why `Move-Item` fails with an error message for every file.

The original script still has one flaw: it only takes *.jpg files into account, but not *.jpeg files. For this reason, you need to ask ChatGPT for rectification.

Prompt: `Move-Item` causes an error: "The process cannot access the file because it is being used by another process." I guess the script needs to close the file before it can move it. Please also process *.jpeg files.

You're correct. The image files should be closed before moving them to avoid the "file in use" error. Additionally, we'll include processing for *.jpeg files as well. Here's the updated script:

```
...
$jpegFiles = Get-ChildItem -Path $sourceDir `
               -Filter *.jpg, *.jpeg
...
```

ChatGPT recognizes its mistake and corrects the script, but immediately inserts a new error: the `-Filter` option expects a single character string as an argument, not an enumeration.

Prompt: `$jpegFiles = Get-ChildItem -Path $sourceDir -Filter *.jpg, *.jpeg` causes an error:

```
Cannot convert System.Object[] to the type System.String required by parameter Filter. Spec:
```

I apologize for the mistake. The `-Filter` parameter only accepts a single string. We can handle both *.jpg and *.jpeg files by using `Get-ChildItem` twice or by using `-Include` with `-Recurse`. Here is the corrected script:

```
...
$jpegFiles = Get-ChildItem -Path $sourceDir `
```

```
...  
-Include *.jpg, *.jpeg -Recurse
```

Only the script delivered in the third attempt works satisfactorily. The `-Recurse` option now used has the additional benefit that PowerShell processes not only the image directory but also all subdirectories. This behavior should actually be self-evident, but was admittedly not required in the original prompt.

What is confusing about this example is the triviality of the errors. While the basic concept of the script is fine on the first attempt, the errors it contains are to a certain extent beginner's mistakes. They undermine trust in the AI tool.

8.1.3 Python: SQLite Application

You want to organize internal data with SQLite in a Python script. You know SQLite as a minimalist library for small database applications, but you've never used SQLite together with Python.

Prompt: I am working on a Python script and want to use SQLite as a local database. Which module should I use? Do I need to install any additional libraries? Can you provide a short Hello-World-sample?

To use SQLite in Python, you can utilize the built-in `sqlite3` module. There's no need to install any additional libraries since `sqlite3` is included with Python's standard library. This module provides a lightweight disk-based database that doesn't require a separate server process and allows access using a nonstandard variant of SQL.

Here's a short "Hello World" example of how to use SQLite with Python:

```
import sqlite3  
conn = sqlite3.connect(':memory:')  
cursor = conn.cursor()  
cursor.execute("""CREATE TABLE hello_world  
                (id INTEGER PRIMARY KEY, message TEXT)""")  
cursor.execute("""INSERT INTO hello_world (message)  
                VALUES ('Hello, World!')""")  
conn.commit()  
cursor.execute('SELECT * FROM hello_world')  
print(cursor.fetchall())  
# Output: [(1, 'Hello, World!')]  
conn.close()
```

This script performs the following steps [more details].

You can modify this example to interact with a file-based database by replacing `:memory:` with a file path, like `example.db`.

ChatGPT provides exactly the right starting point for your own experiments. You can perform a short test run to make sure that the script really does work. Then, try to adapt the script to your needs via AI chat.

We further assume that you use VS Code and GitHub Copilot. Of course, you can use any other comparable combination of editor and code wizard.

Prompt: I want to store a URL, an IP address (IPv4/IPv6) and a timestamp in a table. Can you adapt `hello-world.py` accordingly?

GitHub Copilot then generates new code and displays it in the chat section (not printed here). However, you must manually transfer the code into your script.

An alternative procedure is to select the relevant code section and formulate your code request via `Ctrl` + `I` or `cmd` + `I` in an inline chat. This has the advantage that VS Code now actually suggests changing your code file. You can accept or reject the new code using **Accept** or **Discard**.

Prompt: Please save the database persistent in `log.db`. If this file exists, open it. If not, create a new database with the `web_data` table.

However, the inline chat process isn't really "smart" yet. First, the logic of the code is clearly wrong here.

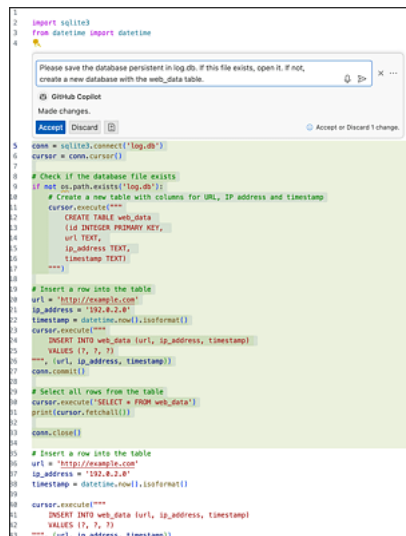


Figure 8.1 GitHub Copilot Didn't Do a Great Job: Faulty New Code Logic and Incomplete Integration into the Existing Code

The test as to whether the database file exists must be carried out *before* the database is opened or created. Second, the new code only partially matches the replaced code. Imports are missing, parts of the code are redundant, and so on. If you accept the AI suggestion, you'll need to edit the code extensively afterwards.

A Look into the Future

This example explains why we prefer a chat function that is separate from the code when making major code changes and why we prefer to carry out the actual code conversion manually. But, of course, it's only a matter of time before code wizards are smart enough to make complex code changes on their own.

8.1.4 Bash Troubleshooting

ChatGPT can also help with troubleshooting. Some time ago, we wanted to process a comma-separated values (CSV) file. The two columns for URL and database were separated by semi-colons. We've tried to split the columns using `cut -d;`, but this fails:

Prompt: The following Bash script doesn't work. Why?

```
tail -n +2 unused.csv | while IFS= read -r line; do
    url=$(echo $line | cut -d; -f1)
    db=$(echo $line | cut -d; -f2)
    echo "url=$url db=$db"
done
```

ChatGPT version 4 and higher recognizes the error immediately. (What about you?)

The issue with your script is likely due to the incorrect handling of the delimiter in the `cut` command. The delimiter needs to be specified properly, and special characters (like the semi-colon `;` in your case) should be quoted to ensure they are interpreted correctly by the shell. Here is how you can fix the code:

```
tail -n +2 unused.csv | while IFS= read -r line; do
    url=$(echo "$line" | cut -d';' -f1)
    db=$(echo "$line" | cut -d';' -f2)
    echo "url=$url db=$db"
done
```

Older GPT versions failed with this example, as did the local models `llama3:8b` and `gemma2:9b`.

8.1.5 Python: Recursive Search for Files

The starting point for this example was the work on a script to recursively process all files with predefined identifiers starting with a start directory (see also [Chapter 10, Section 10.4](#)).

Prompt: Python: I need a function that returns all files in a directory (recursively) with given extensions. I will pass two parameters, directory path and a list of extensions (e.g., `[".js", ".php"]`). The code should run on Linux, macOS, and Linux. Could you please help?

Certainly! Below is a Python function that will search through a directory recursively and return a list of all files that match the given extensions. This function uses the `os` and `os.path` modules for directory traversal.

```
def get_files_with_extensions(directory, extensions):
    matching_files = []
    for root, _, files in os.walk(directory):
        for file in files:
            if any(file.endswith(ext) for ext in extensions):
                matching_files.append(os.path.join(root, file))
    return matching_files

# example usage
print(get_files_with_extensions("/home/kofler",
                                [".js", ".php"]))
```

[more details]

8.1.6 Other Prompt Examples

In this section, we've collected some more prompts from our real-life work, but have spared ourselves the presentation of the results or the discussion of the results. The prompts should simply serve as a suggestion.

Prompt: PowerShell: I need a script to scan my download directory for files older than 30 days. The script lists the files alphabetically (including size information) and offers to delete them after explicit confirmation (input of "yes").

Prompt: Bash: I want to write a script that symmetrically encrypts all files that are passed as parameters. The encryption key (binary) is the content of `/etc/secretkey`. Add `.crypt` to the new file name, and save it in the same directory as the old one. With the `--delete-original` option, delete the original files.

Prompt: Bash: Write a script that uses ImageMagick to create JPEG thumbnails of the front page of all PDF files in a directory. For the thumbnail file names, replace the .pdf extension with .jpg. Skip PDF files that already have a thumbnail.

Prompt: Python: I have a CSV file like this:

[20 lines of CSV text]

I need a script to visualize the column “energy consumption” over time. Can you help?

Prompt: Python/matplotlib: How can I label only every second point on the x-axis?

Prompt: Python: I want to display an image and interactively mark points on it. I need the x and y coordinates of these points. What are my options?